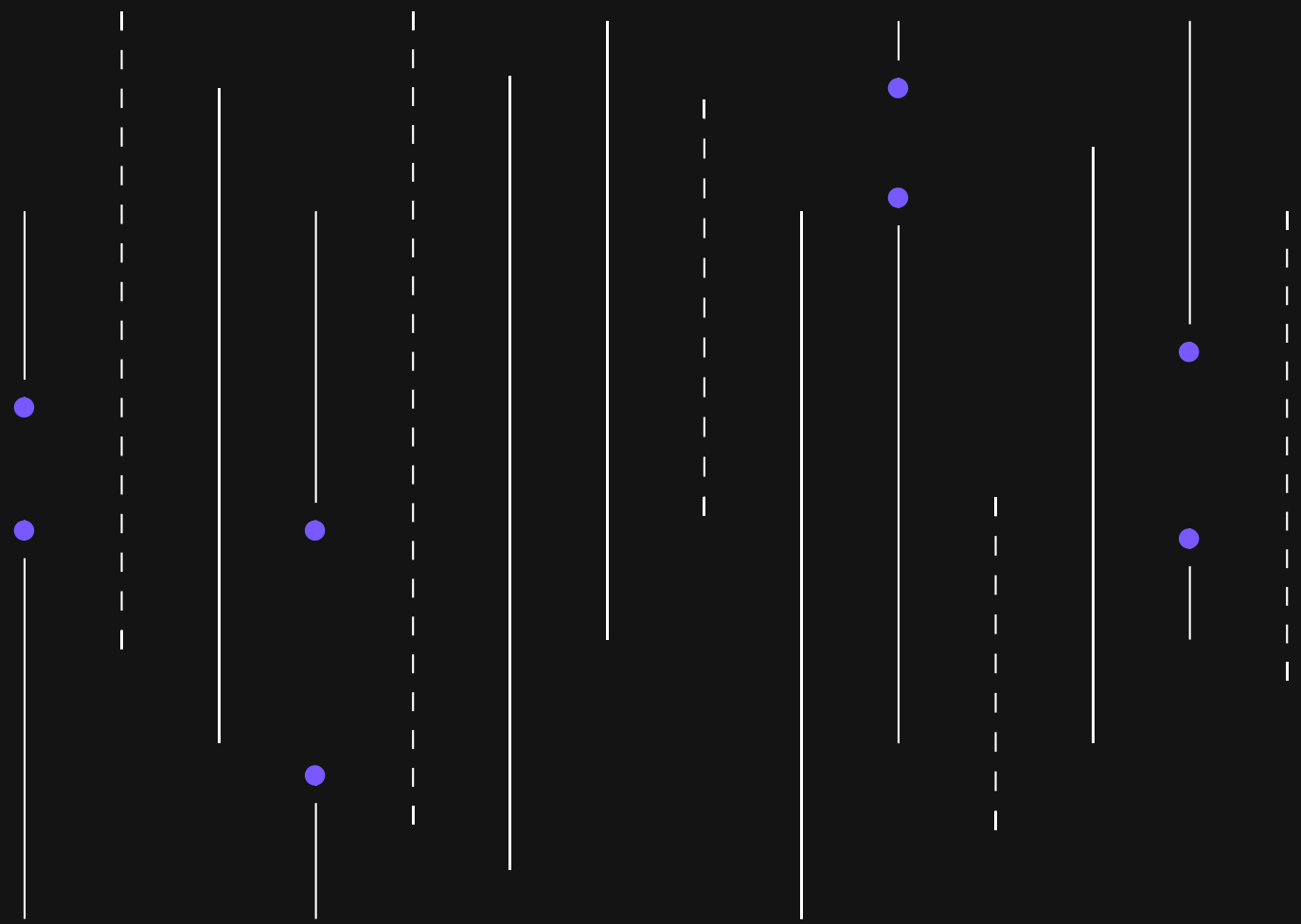
Машинное обучение

Лонгрид 1

Неделя 2

Контроль качества и выбор модели



Содержание

Глава 1. Что такое кросс-валидация 3
1.1. Методы кросс-валидации 4
1.1.1. Отложенная выборка 4
1.1.2. k-Fold 5
1.1.3. Shuffle, group и stratify: управление составом фолдов 7
1.1.4. Leave-One-Out 10
1.1.5. Leave-p-Out 11
1.1.6. Leave-One-Group-Out 11
1.1.7. ShuffleSplit и его варианты: random subsampling CV 12
1.1.8. RepeatedKFold и RepeatedStratifiedKFold 14
1.1.9. Nested-кросс-валидация 14
1.1.10. Bootstrap 16
Глава 2. Оценка на временных рядах 18
2.1. Expanding window 18
2.2. Sliding window 19
2.3. TimeSeriesSplit 19
Глава 3. Практическое применение — функция cross_val_score 20
3.1. Как выбрать подходящий метод валидации 20
3.2. Итого: для чего нужна кросс-валидация 21
Глава 4. Подбор гиперпараметров 22
4.1. Диагностика через кривые: validation curve 23
4.2. Диагностика через кривые: learning curve 24
4.3. GridSearchCV: перебор по сетке параметров 25
4.4. Optuna 27
4.4.1. Запуск Optuna 29
Глава 5. Работа с моделями в sklearn 31
5.1. API sklearn-моделей: fit, predict, predict_proba 31
5.2. Train/test split 31
5.3. Масштабирование: StandardScaler vs MinMaxScaler 32
5.4. Кодирование категориальных признаков 32
5.5. Построение pipeline 33
Если кратко 36

Контроль качества и выбор модели

Контекст

В машинном обучении важен не только тщательный подбор алгоритмов, но и проверка их устойчивости. Кросс-валидация и анализ кривых обучения помогают отличить удачную модель от подстроенной под тренировочные данные, а разделение данных — защитить их.

Применение

Правильная валидация — защита от переобучения и ложных выводов. Методы оценки качества незаменимы в прогнозировании, тестировании и любых задачах с ограниченными данными.

Краткий план

→ Что такое кросс-валидация
→ Оценка на временных рядах
→ Практическое применение — функция cross_val_score
→ Подбор гиперпараметров
→ Работа с моделями в sklearn
→ Если кратко

Глава 1. Что такое кросс-валидация

Качество модели зависит не только от алгоритма, но и от того, насколько корректно организован процесс его оценки. Без правильного разделения данных метрика может быть обманчивой. Например, если мы случайно «подглядываем» в тестовую выборку при обучении, метрики покажут отличные результаты, но на новых данных модель провалится. Такой эффект называется **утечкой данных (Data Leakage)**. Это одна из самых частых и опасных ошибок в машинном обучении.

Поэтому важно чётко разграничивать:

- **обучающую выборку** (англ. train set) — данные, используемые для обучения модели;
- **тестовую выборку** (англ. test set) — данные, используемые только для финальной оценки модели (будто это новые реальные данные).

Ещё лучше — валидировать модели с помощью **кросс-валидации**, которая помогает получить более устойчивую и релевантную оценку качества.

ОПРЕДЕЛЕНИЕ

Кросс-валидация (англ. cross-validation) — это метод оценки качества модели машинного обучения, при котором данные многократно разделяются на обучающую и тестовую выборки, чтобы проверить, насколько хорошо модель обобщает результаты на новых данных.

Кросс-валидация проста в понимании, легко реализуется на практике и, как правило, даёт менее смещённые оценки эффективности по сравнению с другими подходами. Это делает её незаменимым инструментом для обоснованного выбора финальной модели.

Существует множество разновидностей кросс-валидации, но все они строятся на одном базовом алгоритме действий.

1. Разделить датасет на две части: обучающую и тестовую.
2. Обучить модель на обучающей выборке.
3. Оценить её на тестовой выборке.
4. Повторить эти шаги несколько раз в зависимости от выбранного типа кросс-валидации.

Рассмотрим самые полезные и распространённые методы кросс-валидации.

1.1. МЕТОДЫ КРОСС-ВАЛИДАЦИИ

1.1.1. Отложенная выборка

ОПРЕДЕЛЕНИЕ

Отложенная выборка (англ. hold-out) — самый простой и наиболее распространённый подход, который разработчики используют ежедневно. Классическая hold-out-схема — `train_test_split`.

Классический алгоритм отложенной выборки приведён ниже.

1. Случайным образом выборку делят на две части — обучающую и тестовую.
   - Чаще всего используется пропорция 80/20 или 70/30, но на практике допустимы и другие соотношения.
   - До финальной оценки качества модели тестовая выборка никак не используется (например, не для EDA, бейзлайна или экспериментов). После разделения вернуться к тестовой выборке можно только на шаге 3.
2. Обучаем модель на обучающей выборке. Этот шаг включает все манипуляции: EDA, бейзлайн модели, эксперименты и так далее.
3. Оцениваем модель на тестовой выборке.
4. Сохраняем полученные метрики — они дают нам представление о том, как модель будет работать на основе новых данных.

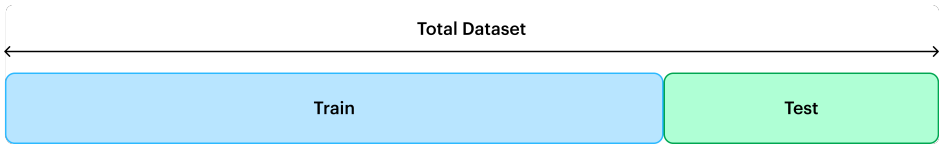


Рис. 1. Hold-out-кросс-валидация. Источник: neptune.ai

Метод hold-out прост в реализации и хорошо подходит для базовой проверки моделей. Однако у него есть важный недостаток — результаты могут быть нестабильными, особенно если выборка мала или классы несбалансированы. Одно неудачное случайное разделение может существенно повлиять на итоговую оценку.

На практике часто используют не два, а три подмножества:

- **обучающая выборка** (train set) — используем для непосредственного обучения модели;
- **валидационная выборка** (validation set) — настраиваем на ней гиперпараметры, выбираем модель, контролируем процесс обучения. Про подбор гиперпараметров поговорим в разделе ниже;
- **тестовая выборка** (test set) — применяем только один раз в самом конце, чтобы честно оценить финальную модель.

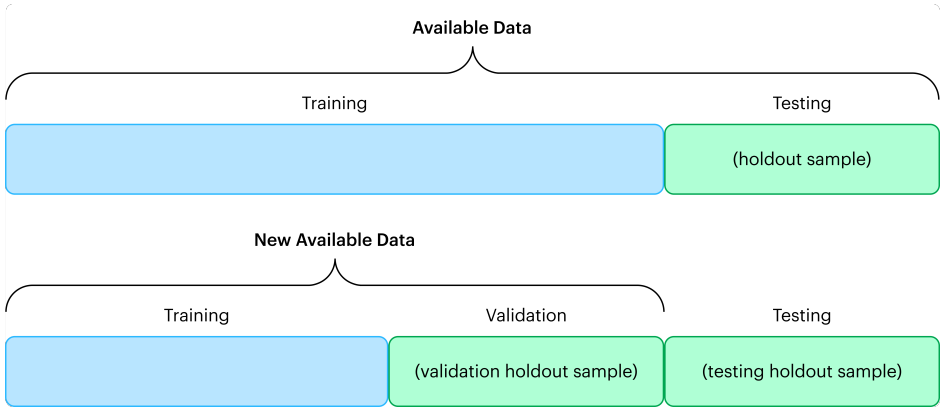


Рис. 2. Разбиение на 3 выборки. Источник: algotrading101.com

Наиболее частый подход

1. Делим данные на train + test (например, 80/20).
2. Часть train отдаём на validation-выборку (столько процентов, сколько считаем нужным).

Применять hold-out стоит в случаях,

- когда выборка достаточно большая и случайное разделение не приводит к значительным перекосам в изначальном распределении;
- необходимо оперативно оценить модель, особенно на ранних этапах;
- требуется отложить независимую тестовую выборку для финальной оценки.

1.1.2. k-Fold

ОПРЕДЕЛЕНИЕ

k-Fold — более надёжная альтернатива простому hold-out, который устраняет его главный недостаток. Модель проверяется несколько раз на случайной части данных и разных подмножествах, что делает итоговую оценку более устойчивой и репрезентативной.

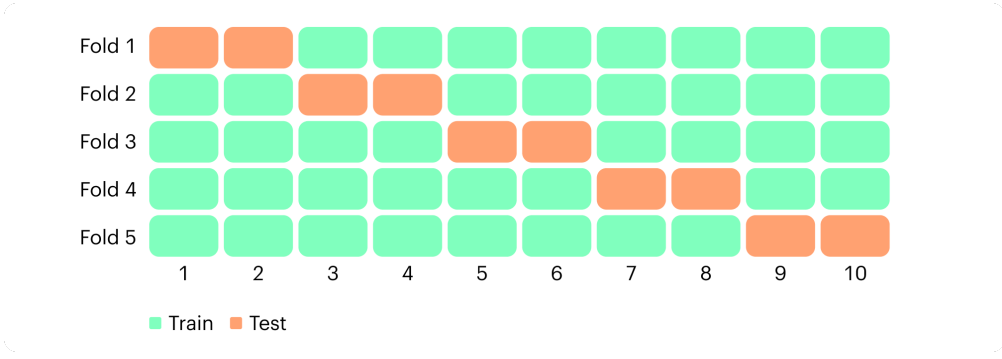


Рис. 3. Схема k-fold кросс-валидации.

Алгоритм k-Fold

1. Выбрать и поделить данные на `k` равных (или почти равных) частей — **фолдов** (часто используют 5 или 10).
2. На каждой итерации:
   - использовать `k-1` фолдов для обучения;
   - оставшиеся — для тестирования.
3. Обучить новую модель с нуля на каждом разбиении.
4. Сохранить метрики валидации (например, accuracy или AUC).
5. Повторить процесс `k` раз, используя новую часть как тестовую.
6. Усреднить все метрики на финальном этапе, чтобы получить обобщённую оценку качества.

При использовании hold-out модель проверяется один раз на фиксированной части данных — они могут быть смещены в одну из сторон. В отличие от него, метод k-Fold:

- тестирует модель на всех частях выборки — каждая строка данных используется и для обучения, и для проверки;
- уменьшает дисперсию оценки, принося более надёжный и устойчивый результат;
- снижает зависимость от конкретного случайного разбиения, что особенно важно при ограниченном объёме данных.

Однако стоит помнить, что при большом `k` нужно обучать много моделей — это может быть долго и ресурсоёмко, особенно если сам датасет объёмный.

Рекомендуется применять k-Fold,

- когда данных не так много и один hold-out может привести к нестабильной оценке;
- хочется получить честную и воспроизводимую оценку, не зависящую от случайностей;
- планируется сравнение различных моделей или подбор гиперпараметров.

> **Примечание.** И хотя hold-out, k-Fold и другие методы — это разные подходы к валидации модели, это не значит, что они полностью взаимоисключаемые. Большинство алгоритмов также могут использоваться совместно в зависимости от цели.

Например, для hold-out и k-Fold на практике часто используют двухуровневую схему.

1. Через hold-out все данные делятся:
   - на train,
   - validation,
   - test.
2. На train и validation применяется k-Fold-кросс-валидация:
   - для подбора гиперпараметров модели,
   - оценки стабильности модели на разных фолдах,
   - выбора лучшего алгоритма для построения модели.
3. На финальном этапе алгоритм с наиболее подходящими параметрами обучается на всём train + validation и финально оценивается на test.

1.1.3. Shuffle, group и stratify: управление составом фолдов

За то, как именно объекты датасета распределяются по фолдам, в sklearn отвечают три независимых механизма:

- **shuffle —** перемешивание;
- **group —** группировка;
- **stratify —** стратификация.

Каждый из них решает свою задачу, и их можно свободно комбинировать друг с другом. Разберём каждый механизм по порядку, а в конце посмотрим, как их можно скомбинировать.

Shuffle

- Если параметр `shuffle = False` (это значение по умолчанию для k-Fold) — фолды формируются строго по порядку следования строк в датасете. Это может быть проблемой, если данные изначально отсортированы, например по классу или по дате.
- При `shuffle = True` — строки случайно перемешиваются перед разбиением на `k` частей, а `random_state` фиксирует конкретное перемешивание для воспроизводимости. При этом каждый объект всё равно попадёт в тестовую часть ровно один раз за полный цикл: перемешивание влияет только на то, какие объекты окажутся вместе в одном фолде.

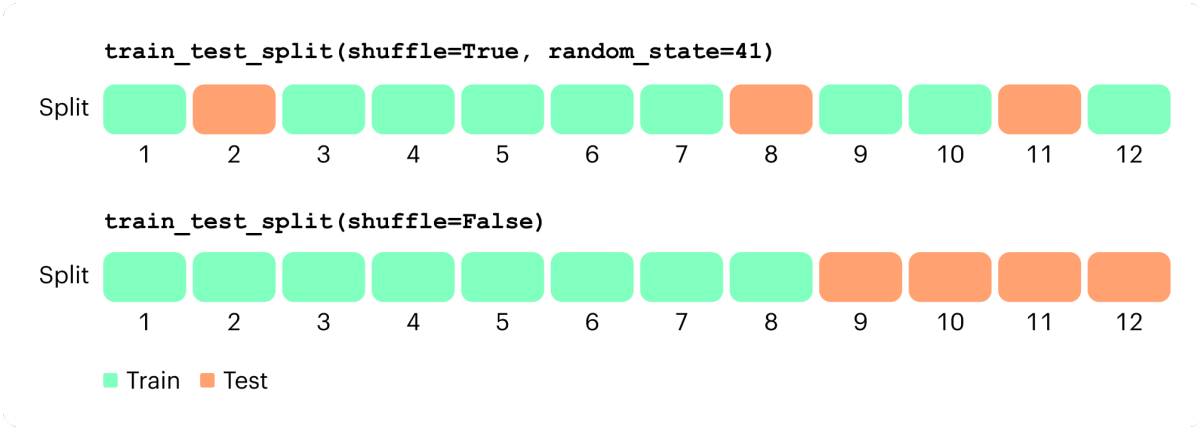


Рис. 4. Разделение данных на выборки с перемешиванием и без.

GroupKFold

Иногда несколько строк данных на самом деле связаны между собой: они принадлежат одному и тому же пациенту, пользователю или магазину. Если такие связанные объекты случайно окажутся одновременно и в обучающей, и в тестовой выборке, модель фактически «увидит» этот объект уже на этапе обучения и будет неправомерно хорошо предсказывать по нему на тесте. Это отдельный вид утечки данных, который называют утечка через группы.

ОПРЕДЕЛЕНИЕ

GroupKFold — вариант k-Fold, в котором разбиение производится не по отдельным объектам, а по целым группам: все строки одной группы гарантированно попадают только в train или только в test одного и того же фолда.

ПРИМЕР

Есть датасет с медицинскими анализами: 100 пациентов, у каждого по 10 визитов, итого 1 000 строк. Обычный `KFold(k = 5)` случайно раскидает по 200 строк

в каждый фолд, и с высокой вероятностью 7 визитов пациента Иванова попадут в train, а 3 оставшихся в test.

Модель уже «видела» физиологию Иванова на этапе обучения, поэтому предсказание по его тестовым визитам получится обманчиво точным, а реальная оценка на новых, ранее не встречавшихся пациентах окажется хуже. Решение простое: группировать разбиение по `patient_id`, чтобы все визиты одного пациента целиком попадали либо в train, либо в test.

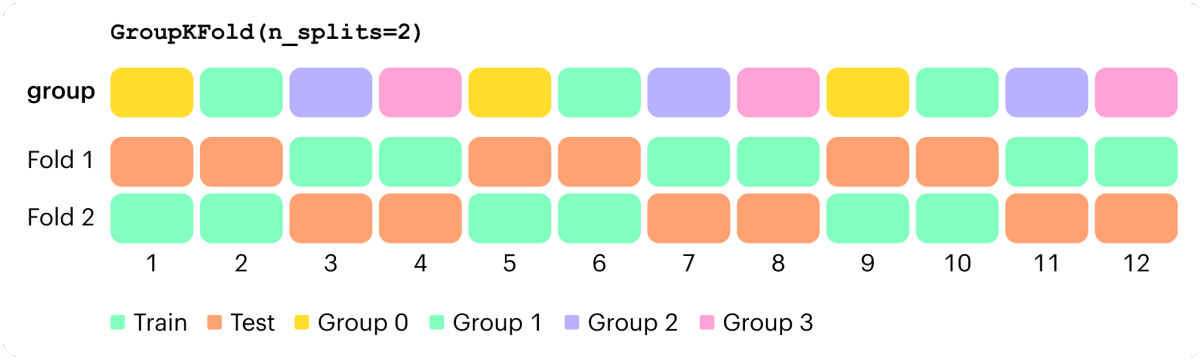


Рис. 5. Схема работы GroupKFold из sklearn.

Каждая группа попадёт в тестовый фолд ровно один раз, а количество уникальных групп должно быть не меньше числа фолдов `n_splits`.

Stratified k-Fold

При сильном дисбалансе целевой переменной обычная k-Fold кросс-валидация часто даёт искажённую оценку качества модели, поскольку при случайном разбиении фолды могут случайно оказаться почти однородными.

ОПРЕДЕЛЕНИЕ

Stratified k-Fold — это улучшенный вариант k-Fold, в котором распределение классов в каждом фолде сохраняется таким же, как и в полном датасете. Если в исходной выборке класс 0 встречается в 70% случаев, а класс 1 — в 30%, то каждый фолд тоже будет содержать примерно 70%/30% этих классов.

Это особенно важно при дисбалансе классов, иначе результаты валидации могут быть нестабильными и вводить в заблуждение.

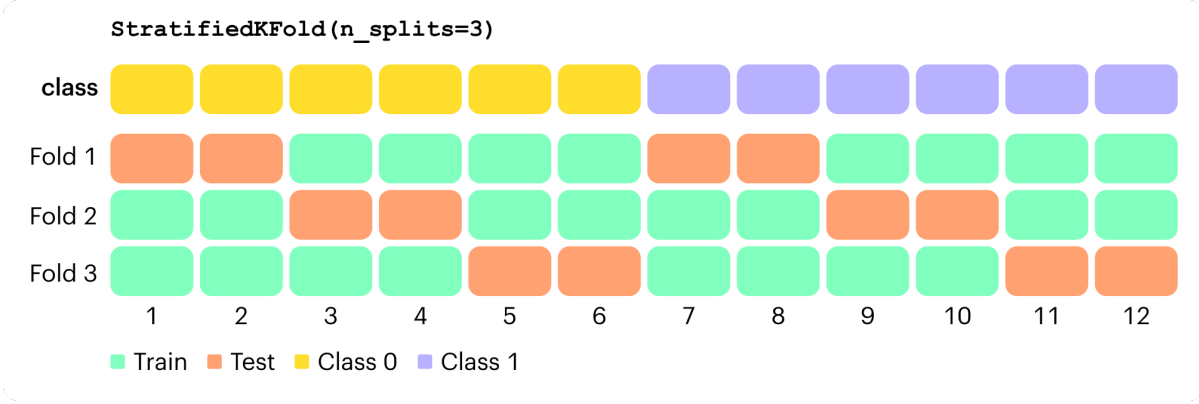


Рис. 6. Схема работы StratifiedKFold из sklearn.

Алгоритм stratified k-Fold

1. Выбрать количество разбиений `k` (обычно 5 или 10).
2. Разделить данные на `k` равных частей **с сохранением исходной пропорции классов в каждом.**
3. На каждой итерации:
   - использовать `k - 1` фолдов для обучения;
   - оставшиеся — для валидации.
4. Обучить модель с нуля на `k - 1` частях.
5. Валидировать оставшиеся части.
6. Сохранить метрику (например, accuracy, AUC или F1-score).
7. Повторить шаги 3–6 `k` раз, меняя валидационный фолд.
8. Усреднить полученные метрики для финальной оценки модели.

Рекомендуется применять stratified k-Fold при дисбалансе целевой переменной. В остальном данный метод идентичен классическому k-Fold.

ПРИМЕР

Обнаружение мошенничества

Предположим, что только 1% наших данных является мошенническим — такое часто бывает, так как мошеннические данные встречаются достаточно редко.

Проблема: обычный k-Fold может дать fold без мошенников.

Решение:

- используем stratified split,
- в каждом фолде сохраняется баланс классов.

StratifiedGroupKFold

Что делать, если одновременно нужно и сохранить группировку, и не потерять баланс классов? Для этого существует комбинированный метод — StratifiedGroupKFold.

ОПРЕДЕЛЕНИЕ

StratifiedGroupKFold — вариант k-Fold, который одновременно учитывает группы (не разбивает их между train и test) и старается сохранить в каждом фолде распределение классов, максимально близкое к исходному, насколько это позволяют границы групп.

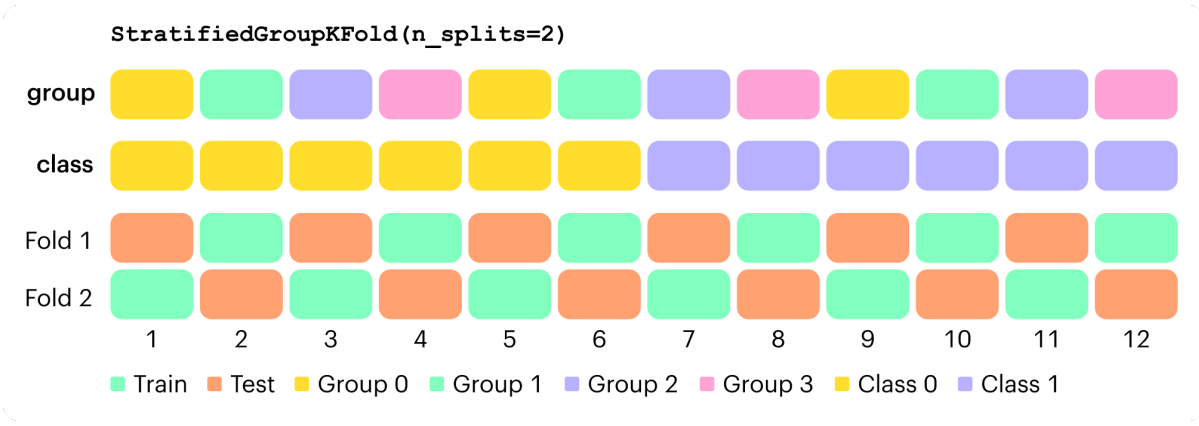


Рис. 7. Схема работы StratifiedGroupKFold из sklearn.

Стоит понимать, что полная стратификация здесь не гарантируется. Если группы сильно разного размера, а классы внутри них распределены неравномерно, метод даёт лишь лучшее возможное приближение к исходным пропорциям, а не точное совпадение.

1.1.4. Leave-One-Out

ОПРЕДЕЛЕНИЕ

Leave-One-Out-кросс-валидация (LOO) — это предельный случай k-Fold-кросс-валидации, в котором число фолдов `k` равно числу объектов `n` в выборке. Это значит, что каждый отдельный объект становится своей собственной тестовой выборкой, а остальные — обучающей.

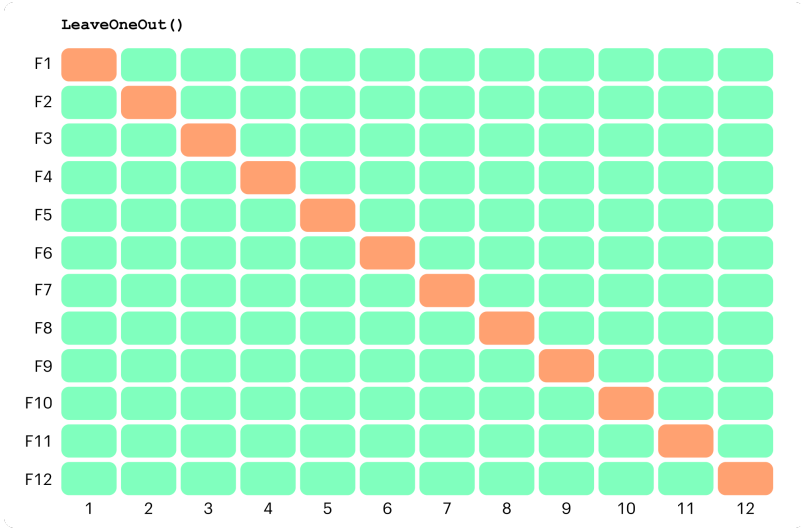


Рис. 8. Схема работы LeaveOneOut из sklearn.

Алгоритм Leave-One-Out

1. Выбрать один объект в качестве тестовой выборки. Все остальные `n - 1` объектов составят обучающую выборку.
2. Обучить модель на этих `n - 1` объектах.
3. Предсказать результат для одного оставшегося объекта.
4. Сохранить метрику.
5. Повторить шаги 1–5 для каждого объекта в датасете.
6. Усреднить метрики по всем итерациям.

Метод Leave-One-Out:

- помогает строить для каждого тестового примера свою модель: её обучают на всех данных, кроме одного объекта — именно он становится тестовым. Таким образом, модель использует почти все доступные данные для обучения, обеспечивая надёжную оценку;
- подходит для задач, в которых изначальная выборка небольшая и важно использовать все данные максимально эффективно.
  Но если имеется 10 000 объектов, придётся обучить 10 000 отдельных моделей, так что затраты по времени и ресурсам для такого метода достаточно высоки.

ПРИМЕР

Маленький медицинский датасет

В датасете 100 пациентов:

- обучаемся на 99,
- тестируем на 1,
- повторяем 100 раз.

Плюс: максимум данных в обучении. Минус: длительное время обучения.

1.1.5. Leave-p-Out

ОПРЕДЕЛЕНИЕ

Leave-p-Out-кросс-валидация — это обобщение метода Leave-One-Out, в котором вместо одного объекта в тестовую выборку помещается сразу `p` объектов.

Вместо теста на одном объекте (например, как в LOO или Leave-p-Out) этот алгоритм тестирует модель на произвольных комбинациях p-объектов, а оставшиеся `n - p` используются для обучения. Следовательно, модель проверяется на всех возможных сочетаниях тестовых подмножеств из `p` элементов.

Например, если $n = 100$, $p = 2$, то потребуется обучить C_n^p 4950 моделей. Стоит обратить внимание, что у этого метода высокая вычислительная сложность, которая будет комбинаторно расти с увеличением `p`. Если $p = 1$, то Leave-p-Out превращается в Leave-One-Out.

1.1.6. Leave-One-Group-Out

Так же как GroupKFold расширяет k-Fold учётом групп, у Leave-One-Out есть свой групповой аналог.

ОПРЕДЕЛЕНИЕ

Leave-One-Group-Out (LOGO) — метод кросс-валидации, в котором на каждой итерации ровно одна целая группа целиком становится тестовой выборкой. А все остальные группы — обучающей.

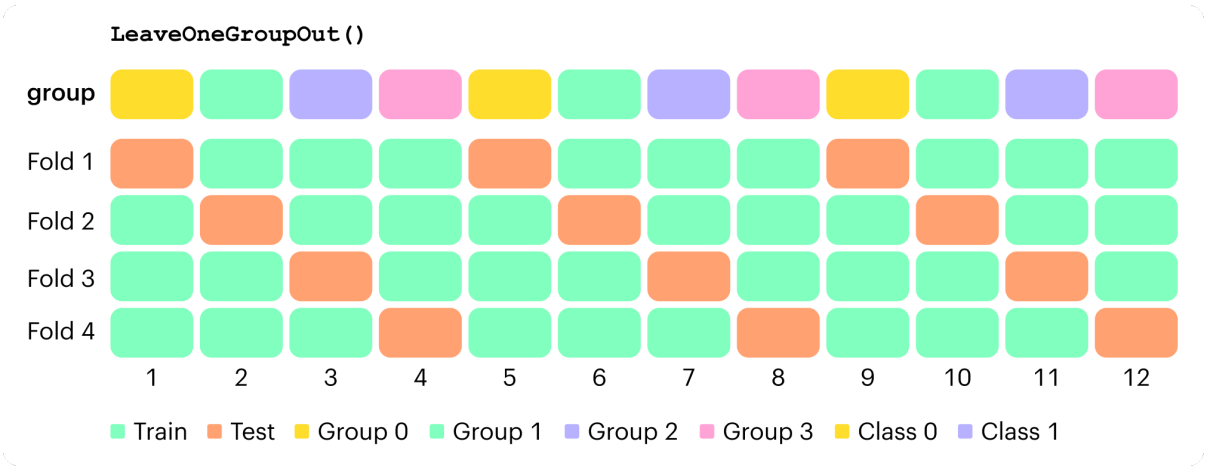


Рис. 9. Схема работы LeaveOneGroupOut из sklearn.

Число итераций совпадает с числом уникальных групп: если групп k, модель обучается k раз. Размер групп обычно неодинаков — например, у разных пациентов разное число визитов). Поэтому итоговую ошибку нельзя усреднять поровну: крупные группы должны вносить больший вклад в оценку.

Финальная метрика считается как взвешенное среднее ошибок по всем группам:

$$e = \frac{1}{m} \sum_{t=1}^{k} m_t \cdot e_t.$$

Здесь e это итоговая ошибка модели, k количество групп, e_t ошибка на группе t (когда именно она была тестовой), m_t размер группы t, а m общее число объектов во всех группах. Такое взвешивание не даёт маленьким группам искусственно завышать или занижать итоговую оценку.

1.1.7. ShuffleSplit и его варианты: random subsampling CV

Все рассмотренные выше методы (`k-Fold, GroupKFold, StratifiedKFold`) объединяет одна особенность: они делят выборку на фиксированные, заранее заданные фолды, и каждый объект участвует в тестировании ровно один раз за полный цикл. Существует и другой подход к многократному разбиению данных — random subsampling.

ОПРЕДЕЛЕНИЕ

Random subsampling CV (ShuffleSplit) — метод кросс-валидации, в котором данные многократно и независимо случайным образом разделяются на обучающую и тестовую выборки. В отличие от K-Fold, где размер тестовой выборки определяется числом фолдов, в ShuffleSplit размер тестовой выборки и количество разбиений выбираются независимо.

Алгоритм простой: k раз подряд случайно выбираем отложенную выборку и усредняем ошибки, вычисленные на каждом из полученных разбиений. Важно, что здесь **термин «сплит» не то же самое, что «фолд»:** в отличие от k-Fold, где k фолдов вместе покрывают весь датасет ровно один раз, при ShuffleSplit один и тот же объект может оказаться в тестовой выборке несколько раз, а может не попасть туда ни разу.

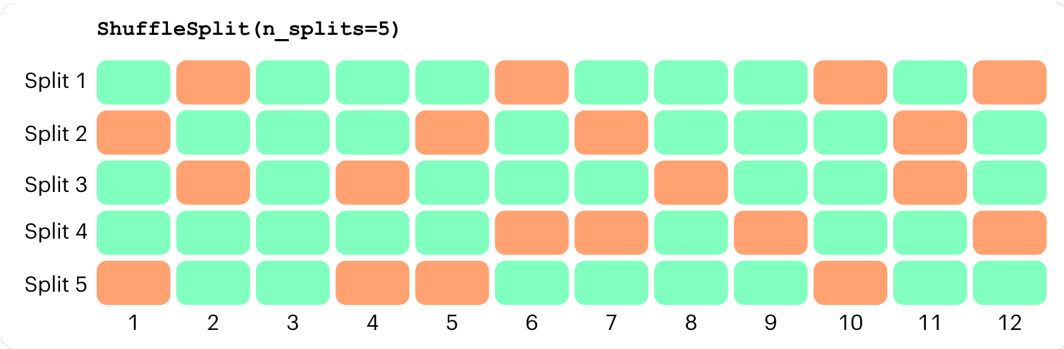


Рис. 10. Схема работы ShuffleSplit из sklearn.

	K-Fold CV	Random subsampling CV (ShuffleSplit)
Разбиение	Делит данные на k фиксированных частей	Каждый раз делает новое случайное разбиение
Тестовая выборка	Каждый объект попадает в тест ровно один раз (за один цикл)	Один объект может попасть в тест несколько раз или ни разу
Данные для тестирования	Все данные используются для тестирования	Только часть данных может быть использована для теста

GroupShuffleSplit

ОПРЕДЕЛЕНИЕ

GroupShuffleSplit — версия ShuffleSplit, учитывающая группировку: объекты одной группы всегда целиком попадают либо в train, либо в test, а сами группы каждый раз случайно перераспределяются между train и test с заданной пропорцией.

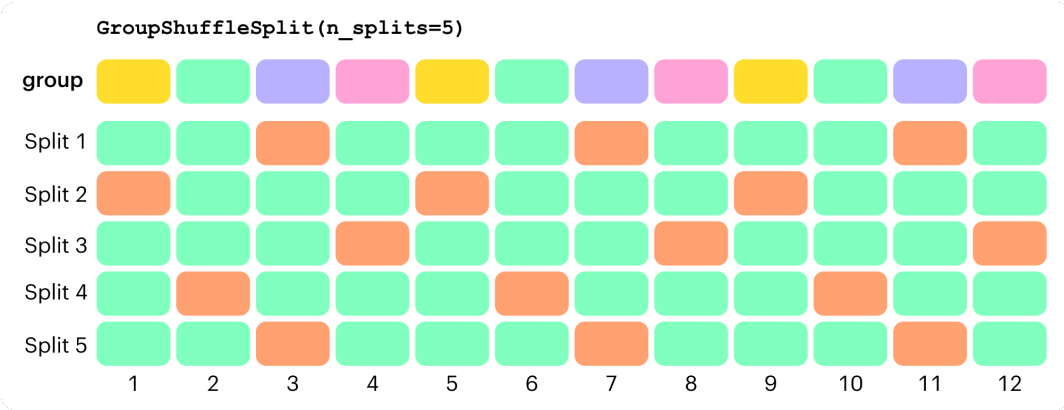


Рис. 11. Схема работы GroupShuffleSplit из sklearn.

В отличие от GroupKFold, где каждая группа ровно один раз попадает в тест, GroupShuffleSplit на каждой итерации заново случайно выбирает заданное число или долю тестовых групп. Поэтому тестовые наборы могут пересекаться, а некоторые группы могут ни разу не попасть в тест.

StratifiedShuffleSplit

ОПРЕДЕЛЕНИЕ

StratifiedShuffleSplit — версия ShuffleSplit, сохраняющая в каждой случайной выборке исходное соотношение классов.

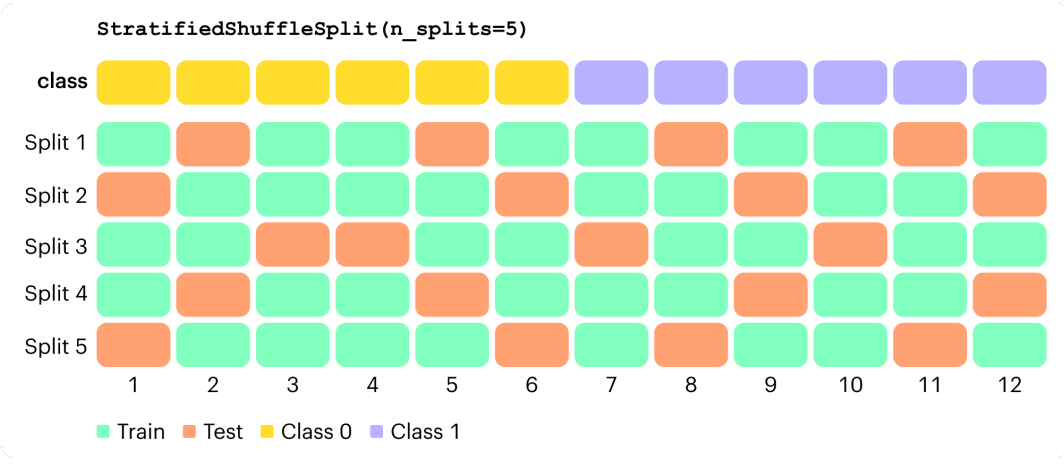


Рис. 12. Схема работы StratifiedShuffleSplit из sklearn.

Разбиение выполняется случайно, но с контролем долей классов на каждой итерации. Это полезно, например, при ограниченном бюджете на эксперименты, когда важно гарантировать нужное количество представителей редкого класса в каждом тестовом наборе, а не рассчитывать на удачу при обычном ShuffleSplit.

1.1.8. RepeatedKFold и RepeatedStratifiedKFold

Ещё один способ увеличить устойчивость оценки, оставаясь в рамках классического k-Fold, состоит в том, чтобы повторить всю процедуру несколько раз с разным случайным перемешиванием.

ОПРЕДЕЛЕНИЕ

RepeatedKFold — данные случайно разбиваются на k фолдов, вся процедура кросс-валидации повторяется `n_repeats` раз с новым перемешиванием, а итоговые метрики усредняются по всем прогонам. Аналогично работает RepeatedStratifiedKFold, дополнительно сохраняя баланс классов в каждом разбиении.

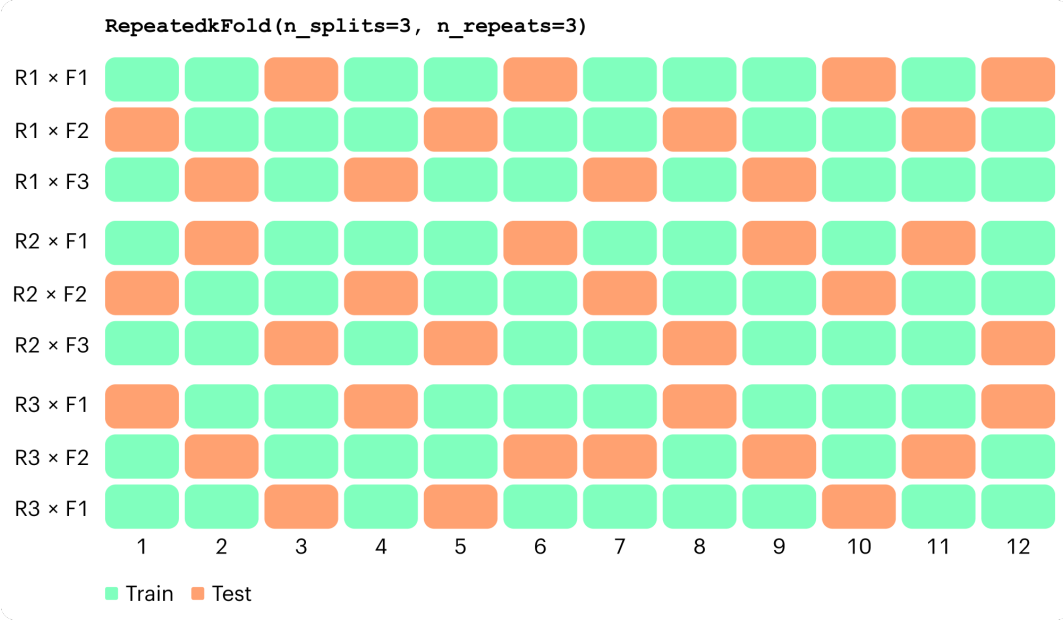


Рис. 13. Схема работы RepeatedKFold из sklearn.

По сути это способ получить более надёжную оценку дисперсии качества модели, не переходя к дорогостоящему Leave-One-Out: вместо того чтобы делать k очень большим, можно оставить умеренное k, но повторить разбиение несколько раз с новым перемешиванием.

1.1.9. Nested-кросс-валидация

Nested-кросс-валидация применяется в случаях, когда необходимо одновременно подобрать гиперпараметры модели (что такое гиперпараметры рассмотрим подробнее в разделах ниже) и корректно оценить её итоговое качество.

Для этого в nested-кросс-валидации существуют:

- **Внешний цикл (outer loop)** — предназначен для оценки обобщающей способности модели. Этот цикл:
  - разбивает данные на `train_outer` и `test_outer`;
  - тестирует итоговую модель только на `test_outer`, который не использовался при подборе гиперпараметров.
- **Внутренний цикл** (inner loop) — реализует поиск оптимальных гиперпараметров или признаков на `train_outer` внешнего цикла.

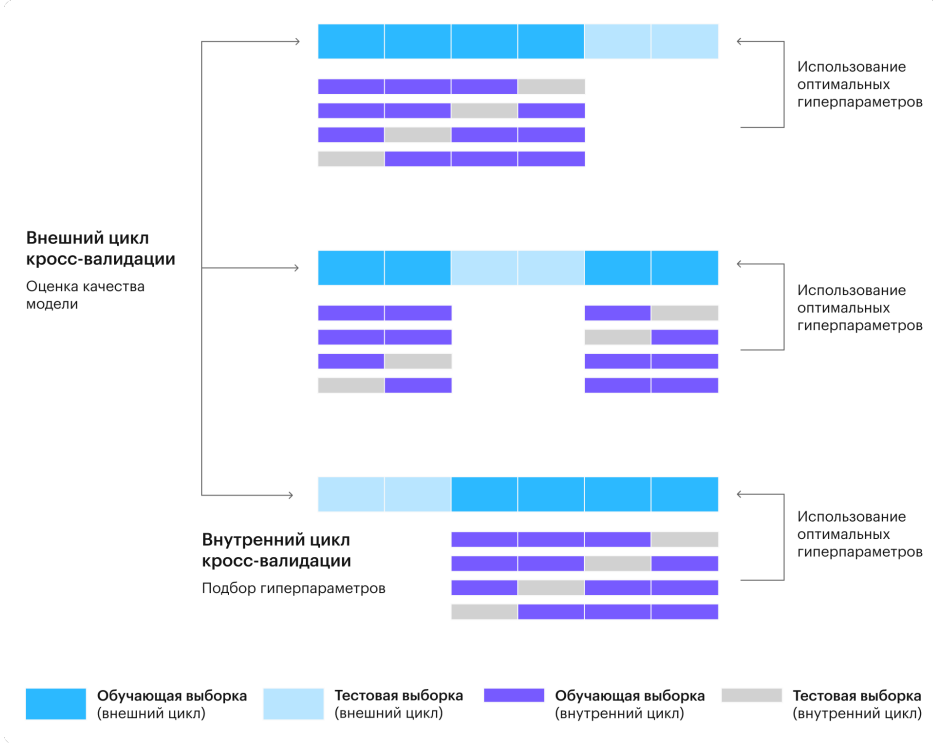


Рис. 14. Nested-кросс-валидация. Источник: stats.stackexchange.com

Алгоритм nested-кросс-валидации

1. Определить множество гиперпараметров `C` для подбора.
2. Поделить данные на `K` внешних разбиений (они могут быть стратифицированы).
3. Для текущего разбиения `k` из `K` (внешний цикл):
   - Пометить фолд `k` как внешний тест.
   - Для комбинации параметров `c` из `C` для каждой части `k_1` из оставшихся `K - 1` (внутренний цикл):
     - оставить `k_1` для валидации;
     - тренировать модель на оставшихся `k - 2` фолдах;
     - проверить качество модели на `k_1`.
   - Посчитать среднее качество по всем `K - 2` итерациям для комбинации параметров `c`.
   - Тренировать модель на `K - 1` наборах, используя ту комбинацию гиперпараметров, которая выбила лучшее значение метрики во внутреннем цикле.
   - Проверить итоговое качество модели на фолде `k`.
4. Усреднить результаты по всем `K` циклам.

С помощью nested-кросс-валидации можно:

- получить реалистичную оценку качества модели,
- предотвратить утечки данных,
- избежать переобучения на этапе подбора гиперпараметров.

Но стоит помнить, что данный метод:

- затратен по времени — если внешний и внутренний `k = 5` и у тебя 10 наборов гиперпараметров, будет обучено 250 моделей ($5 \times 5 \times 10 = 250$);
- не поддерживается в sklearn «из коробки».

1.1.10. Bootstrap

Помимо кросс-валидации, есть и другие методы, которые призваны облегчить и улучшить процесс оценки качества моделей. Один из них — bootstrap.

ОПРЕДЕЛЕНИЕ

Bootstrap — это техника генерации новых выборок из исходных данных путём случайного отбора объектов с возвращением.

Bootstrap помогает оценить дисперсию модели и построить доверительные интервалы для любой метрики (например, accuracy, ROC-AUC или MAE).

С помощью bootstrap вместо одной точечной оценки (например, accuracy = $0{,}83$), можно получить accuracy $\in [0{,}80,\ 0{,}86]$ с вероятностью в 95%, что позволяет говорить о более надёжной модели.

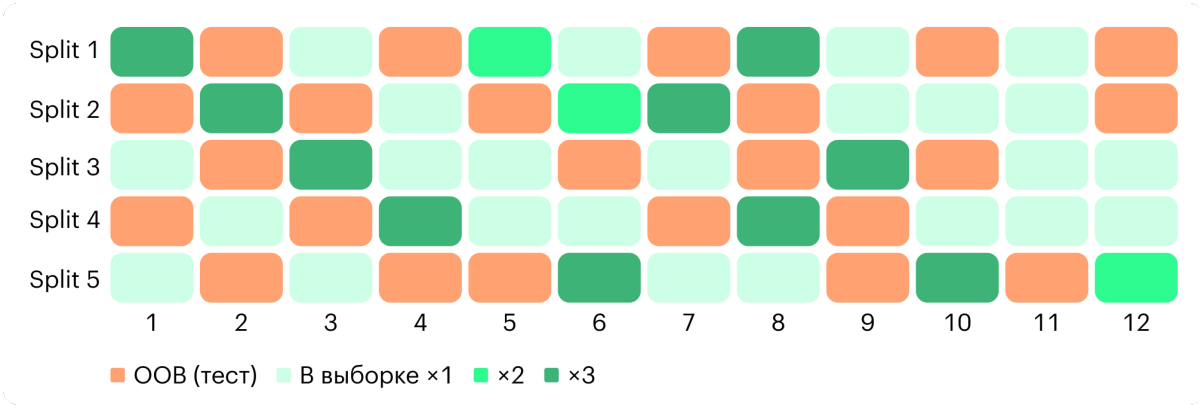


Рис. 15. Пример кросс-валидации методом bootstrap.

Алгоритм bootstrap

1. Сформировать bootstrap-выборку: случайно выбрать m объектов с возвращением, где m равно размеру исходной выборки. Некоторые объекты попадут в неё несколько раз, а часть вообще не попадёт.
2. Обучить модель на полученной bootstrap-выборке.
3. Протестировать модель на объектах, которые не попали в bootstrap-выборку. Такие объекты называют **Out-of-Bag (OOB)**, и именно они выступают тестовой выборкой для этой итерации.
4. Повторить шаги 1–3 много раз (обычно 1000+), каждый раз с новой bootstrap-выборкой, и сохранить метрику на OOB-объектах.
5. Получить распределение метрики, из которого можно получить:
   - оценку качества (среднее);
   - анализ устойчивости решения (СКО);
   - доверительный интервал (перцентили).

Доля объектов, которые не попадают в bootstrap-выборку и естественным образом формируют OOB, при большом m стремится к постоянной величине:

$$\left(1 - \frac{1}{m}\right)^m \approx e^{-1} \approx 0{,}37.$$

Другими словами, около 37% исходных данных не попадают в bootstrap-выборку, а оставшиеся ~63% (с повторами) используются для обучения.

Важные особенности метода:

- модель обучается на выборке того же объёма, что и итоговая (обученная на всех доступных данных), но с повторами и без части объектов.
- с точки зрения распределения, bootstrap-выборка похожа на исходную — сохраняет её основные статистические свойства, хотя и содержит дубликаты и покрывает не все объекты.

Глава 2. Оценка на временных рядах

ОПРЕДЕЛЕНИЕ

Временные ряды — это последовательные наблюдения, для которых важен порядок наблюдений. Работая с такими данными, нельзя нарушать их структуру, а соответственно, и использовать традиционные подходы.

Например, k-Fold случайно перемешивает данные. В случае временных рядов, это приводит к их утечке из будущего в прошлое, приводя к бессмысленному результату.

Самый базовый подход, который можно использовать — последовательный hold-out, когда данные выборки разбиваются по временной шкале:

- train — ранние периоды (например, 2020–2022 годы).
- validation — промежуточные данные (например, 2023 год).
- test — самые последние наблюдения (например, 2024 год).

Это защищает от «подглядывания в будущее». Однако, как мы уже знаем, оценки по одному разбиению недостаточно для реалистичной валидации модели, поэтому для временных рядов в основном применяют другие методы.

2.1. EXPANDING WINDOW

ОПРЕДЕЛЕНИЕ

Нарастающее окно (англ. expanding window) — способ валидации модели, в котором на каждом шаге Train-окно расширяется, а в Validation всегда попадает в следующий временной блок. Такой метод подходит для задач, где важно учитывать весь исторический контекст.

ПРИМЕР

У нас есть данные, охватывающие промежуток с 2019 по 2022 год. Следуя логике нарастающего окна, их можно разбить на три фолда.

- Fold 1: train [2019], val [2020].
- Fold 2: train [2019–2020], val [2021].
- Fold 3: train [2019–2021], val [2022].

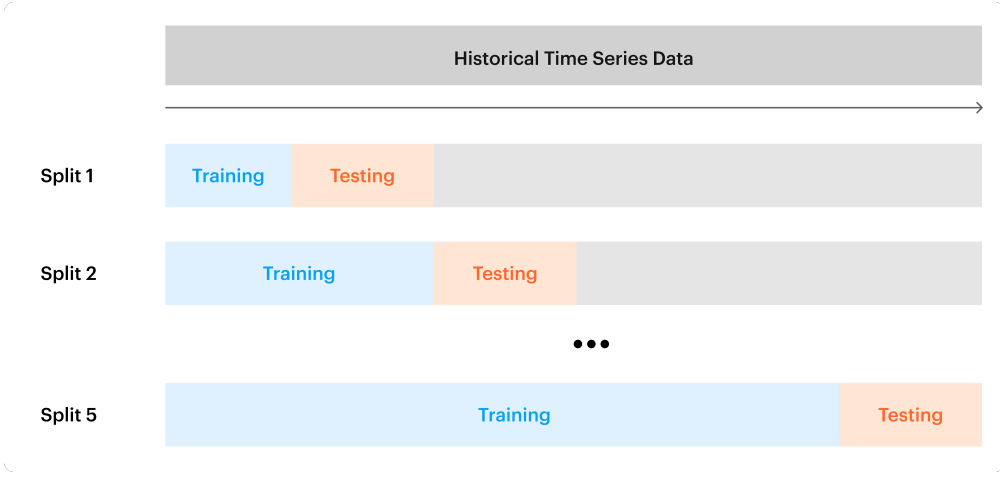


Рис. 16. Визуализация expanding window. Источник: researchgate.net

2.2. SLIDING WINDOW

ОПРЕДЕЛЕНИЕ

Скользящее окно (англ. sliding window) — ещё один метод работы при валидации на временных рядах, когда Train-окно фиксировано по размеру. Однако оно «скользит» вперёд по временной шкале вместе с валидацией. Такой метод подходит для случаев, в которых старые данные со временем теряют актуальность.

ПРИМЕР

У нас есть данные, охватывающие промежуток с 2019 по 2023 год. Следуя логике скользящего окна, подобные данные можно разбить на следующие фолды.

- Fold 1: train [2019–2020], val [2021].
- Fold 2: train [2020–2021], val [2022].
- Fold 3: train [2021–2022], val [2023].

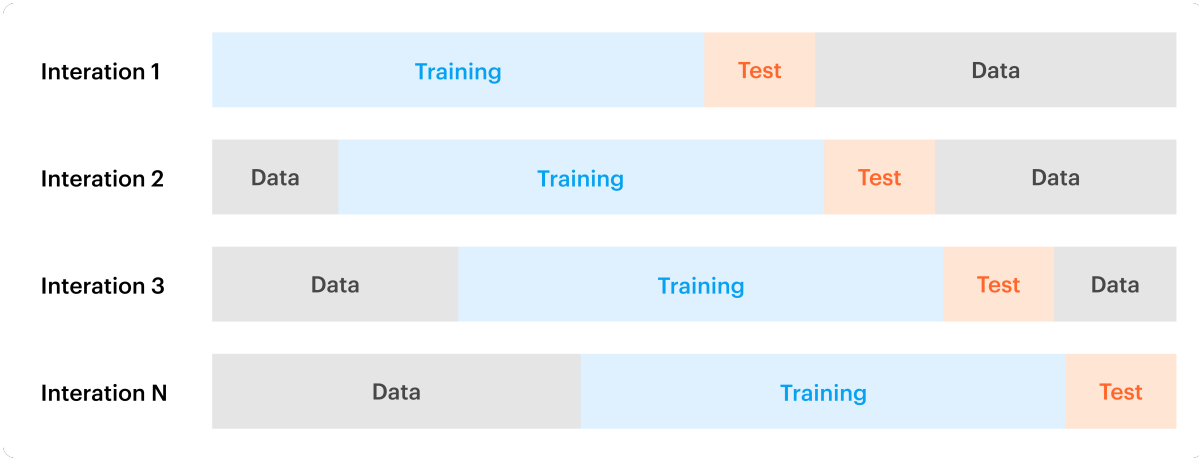


Рис. 17. Визуализация sliding window. Источник: researchgate.net

ПРИМЕР

Прогноз температуры:

- обучаемся на последних 30 днях,
- двигаем окно.

2.3. TIMESERIESSPLIT

С практической точки зрения, для работы с временными рядами удобно пользоваться функцией TimeSeriesSplit из sklearn. Она позволяет реализовать различные вариации expanding window, а именно:

- разделить данные по времени без перемешивания,
- задать желаемое количество фолдов и шаг разбиения.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=3)
for train_index, val_index in tscv.split(data):
    print("TRAIN:", train_index, "VAL:", val_index)
```

Глава 3. Практическое применение — функция cross_val_score

Всё, что мы разобрали выше (hold-out, k-Fold и его варианты, ShuffleSplit-семейство) в sklearn подключается к любой модели буквально одной строкой кода через функцию `cross_val_score`.

```
from sklearn.model_selection import cross_val_score, ShuffleSplit
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression()
cv = ShuffleSplit(n_splits=3, test_size=0.1, train_size=None, random_state=None)

cross_val_score(logreg, X, y, cv=cv)
```

По умолчанию функция использует стандартную метрику качества, заданную для модели (например, accuracy для классификации), но при необходимости можно передать другую через параметр scoring. Есть и родственная функция cross_validate: она дополнительно сохраняет время обучения и время предсказания на каждом прогоне, что удобно, когда важно сравнивать модели не только по качеству, но и по скорости.

3.1. КАК ВЫБРАТЬ ПОДХОДЯЩИЙ МЕТОД ВАЛИДАЦИИ

Мы разобрали много разных схем кросс-валидации, и на практике возникает закономерный вопрос: какую же выбрать для конкретной задачи? Удобно ориентироваться по двум независимым признакам данных:

- есть ли в них группы (связанные между собой объекты);
- есть ли дисбаланс классов.

Для данных с временной структурой действует отдельное правило вне зависимости от групп и классов: обычное перемешивание здесь недопустимо, нужен TimeSeriesSplit, о котором мы говорили в разделе про временные ряды.

Тип разбиения	Без группировки	С группировкой
Без стратификации	train_test_split, KFold, ShuffleSplit, RepeatedKFold, LeaveOneOut, TimeSeriesSplit	GroupKFold, GroupShuffleSplit, LeaveOneGroupOut
Со стратификацией	StratifiedKFold, StratifiedShuffleSplit	StratifiedGroupKFold

Несколько практических ориентиров, которые помогают быстро сориентироваться в этой карте:

- если данных много и группировка не нужна — обычно достаточно KFold или ShuffleSplit, они просты и быстро считаются;
- если в данных есть дисбаланс классов — стоит взять стратифицированный вариант. Без стратификации редкий класс может случайно не попасть в один из фолдов целиком, и метрика на этом фолде окажется бессмысленной;
- если объекты связаны между собой, например визиты одного пациента, действия одного пользователя или история одного магазина, — группировка обязательна, иначе возникает утечка данных через группы;
- если нужны повторные случайные hold-out-разбиения по группам, подходит GroupShuffleSplit: он позволяет отдельно задать число разбиений и число или долю тестовых групп;
- если данные с явной временной структурой — k-Fold и его варианты не подходят в принципе, потому что перемешивание приводит к утечке из будущего в прошлое.

3.2. ИТОГО: ДЛЯ ЧЕГО НУЖНА КРОСС-ВАЛИДАЦИЯ

- **Оценивать качество модели** — чтобы понять, насколько хорошо модель обобщает результаты на данных, не использованных при обучении.
- **Настраивать гиперпараметры** — выбирать лучшие значения параметров модели (подробнее в следующем разделе).
- **Сравнивать и выбирать модели —** сравнивать разные алгоритмы между собой на одинаковых разбиениях данных.
- **Оценивать полный ML-пайплайн —** включая все этапы предобработки данных, а не только саму модель.

Глава 4. Подбор гиперпараметров

Всё, что мы разбираем в этом лонгриде, связано с поздними этапами ML lifecycle и достаточно тесно переплетается между собой. Например, техники кросс-валидации, описанные выше, регулярно используются при подборе гиперпараметров модели.

Вспомним, что мы уже изучали на курсе «Введение в искусственный интеллект».

ОПРЕДЕЛЕНИЕ

Гиперпараметры (англ. *hyperparameters*) — это внешние параметры модели, задаваемые до начала обучения, не изменяемые в его процессе и подбираемые экспертно или с помощью специальных методов оптимизации. Они определяют процесс обучения и влияют на качество модели.

Они задаются до обучения и могут как сильно улучшить итоговое качество, так и ухудшить его (например, привести к переобучению). У каждой модели свой список гиперпараметров (например, learning rate, регуляризация, число деревьев или глубина).

Гиперпараметры не стоит путать с обычными параметрами модели (model parameters): вторые настраиваются автоматически в процессе обучения, например веса линейной регрессии или структура построенных деревьев, тогда как гиперпараметры задаются извне и обучение изменить их не может.

Тогда с учётом перебора гиперпараметров, пайплайн кросс-валидации может выглядеть так.

1. Через hold-out все данные делятся:
   - на train,
   - validation,
   - test.
2. На train и validation применяется кросс-валидация:
   - для подбора гиперпараметров модели,
   - оценки стабильности модели на разных фолдах,
   - выбора лучшего алгоритма для построения модели.
3. На финальном этапе алгоритм с наиболее подходящими параметрами обучается на всём train + validation и финально оценивается на test.

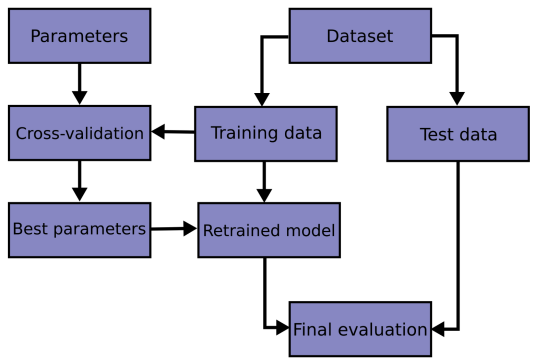


Рис. 18. Схема применения кросс-валидации для подбора гиперпараметров и итогового обучения модели.
Источник: scikit-learn.org

Что касается перебора гиперпараметров: один из основных способов подбора гиперпараметров — это ручной. Он помогает менять параметры от эксперимента к эксперименту, основываясь на опыте или внутренней интуиции. Однако такая концепция делает ручной тюнинг субъективным, плохо масштабируемым методом. После множества итераций можно подобрать устраивающие значения, но проще и быстрее воспользоваться инструментами автоматизации.

Метод	Принцип работы	Поддержка early stopping	Параллелизм	Гибкость условий (nested params)	Поддержка логинга/визуализации	Где применим	Плюсы	Минусы
Grid search	Полный перебор по сетке	Нет	Да (ограничено)	Нет	Частично (`GridSearchCV`)	Маленькие пространства, baseline	Простота, воспроизводимость	Эксп. рост времени при увеличении числа параметров
Randomized search	Случайный выбор комбинаций	Нет	Да	Нет	Частично	Большие пространства, быстрый прототип	Быстрее, чем grid, при ограниченном бюджете	Риск неоптимального разбиения
Hyperopt	Байесовская оптимизация (TPE)	Частично	Да (через Spark или Ray)	Да	Через сторонние библиотеки	Модели со сложными иерархиями параметров	Эффективен, гибок, open source	Относительно сложный метод с устаревшим интерфейсом и ручной визуализацией
Optuna	Байесовская оптимизация + pruning	Да	Да (через Joblib, Dask или Ray)	Да	Да	Современные ML-задачи, Kaggle, продакшен	Поддержка early stopping, многофункционален, гибок	Требует начальной настройки

Как выбрать метод?

- GridSearchCV/RandomizedSearchCV — подойдут для начала разработки и формирования Baseline.
- Optuna — наиболее оптимальный выбор из имеющихся. Он оперативный, высокопроизводительный и с продуманным UI.
- Hyperopt — устаревший и менее гибкий предшественник Optuna, построенный на схожих принципах.

4.1. ДИАГНОСТИКА ЧЕРЕЗ КРИВЫЕ: VALIDATION CURVE

Прежде чем запускать полноценный перебор параметров, полезно увидеть, как вообще качество модели зависит от конкретного гиперпараметра. Для этого существует специальный диагностический инструмент.

ОПРЕДЕЛЕНИЕ

Validation curve (кривая валидации) — график зависимости качества модели от значения **одного гиперпараметра**, построенный отдельно для обучающей выборки и для кросс-валидации.

```python
from sklearn.model_selection import validation_curve

train_scores, test_scores = validation_curve(
    knn, X, y,
    param_name="n_neighbors",
    param_range=range(1, 21),
    cv=5,
)
```

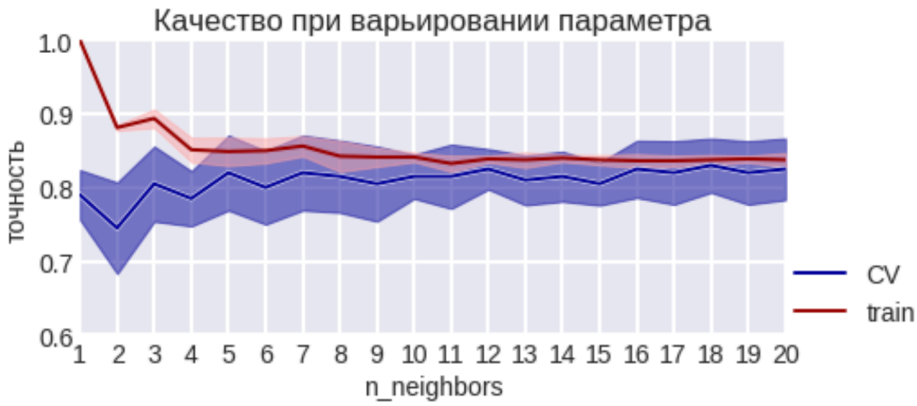


Рис. 19. График validation curve.

Как читать график:

- кривая train — показывает качество на обучающей выборке;
- кривая CV — показывает более честную, реалистичную оценку на кросс-валидации.

Большой разрыв между ними служит верным признаком переобучения. На примере с n_neighbors при k=1:

- модель полностью переобучается (train-качество достигает 1.0, а CV заметно ниже);
- оптимальное значение n_neighbors выбирают по максимальному значению CV-метрики, а разрыв между train- и CV-кривыми используют для диагностики переобучения

4.2. ДИАГНОСТИКА ЧЕРЕЗ КРИВЫЕ: LEARNING CURVE

Второй похожий инструмент отвечает на другой вопрос: не «какое значение параметра выбрать», а «поможет ли модели больше данных».

ОПРЕДЕЛЕНИЕ

Learning curve (кривая обучения) — график, показывающий, как качество модели растёт с увеличением объёма обучающей выборки.

```python
from sklearn.model_selection import learning_curve

train_sizes, train_scores, test_scores = learning_curve(knn, X, y, cv=5)
```

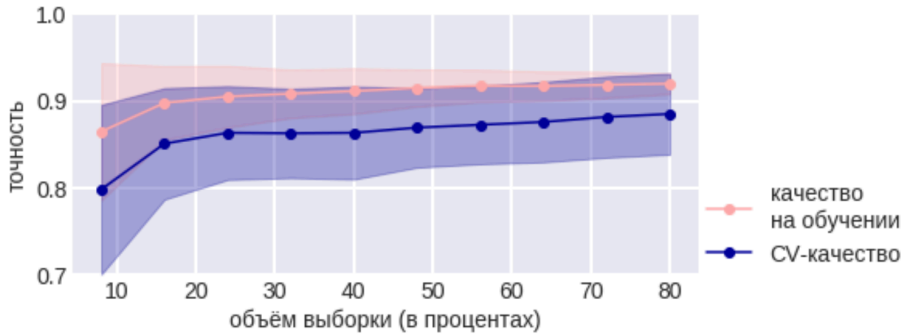


Рис. 20. График learning curve.

- Если кривые обучения и валидации сходятся друг к другу, значит, модель стабилизировалась, добавление новых данных вряд ли даст существенный прирост качества.
- Если же разрыв между кривыми обучения и валидации остаётся большим даже при увеличении объёма выборки, это свидетельствует о переобучении (overfitting). В этом случае могут помочь увеличение объёма обучающих данных, упрощение модели, усиление регуляризации, подбор гиперпараметров.
- Если обе кривые сходятся, но на низком уровне качества, это указывает на недообучение (underfitting). В таком случае обычно помогают использование более сложной модели, создание более информативных признаков, уменьшение регуляризации, более тщательная настройка гиперпараметров.

4.3. GRIDSEARCHCV: ПЕРЕБОР ПО СЕТКЕ ПАРАМЕТРОВ

Когда качество зависит сразу от нескольких гиперпараметров одновременно, кривые валидации уже не позволяют увидеть полную картину, и здесь нужен систематический перебор комбинаций.

ОПРЕДЕЛЕНИЕ

GridSearchCV — инструмент scikit-learn, который перебирает все комбинации значений из заданной сетки гиперпараметров и для каждой из них измеряет качество через кросс-валидацию.

Алгоритм работы можно описать в три шага.

1. Задаём сетку: список значений для каждого гиперпараметра.
2. Для каждой комбинации параметров обучаем модель и замеряем качество через k-Fold кросс-валидацию.
3. Возвращаем лучшую комбинацию, ту, у которой среднее CV-качество оказалось наивысшим.

```python
from sklearn.model_selection import GridSearchCV

parameters = {
    'metric': ('euclidean', 'manhattan', 'chebyshev'),
    'n_neighbors': [1, 3, 5, 7, 9, 11],
}

clf = GridSearchCV(estimator, parameters, cv=5, scoring='roc_auc')
clf.fit(X, y)
clf.cv_results_['mean_test_score']
```

Ниже на рисунке наглядно видна разница между полным перебором сетки (GridSearch) и случайным перебором (RandomizedSearch).

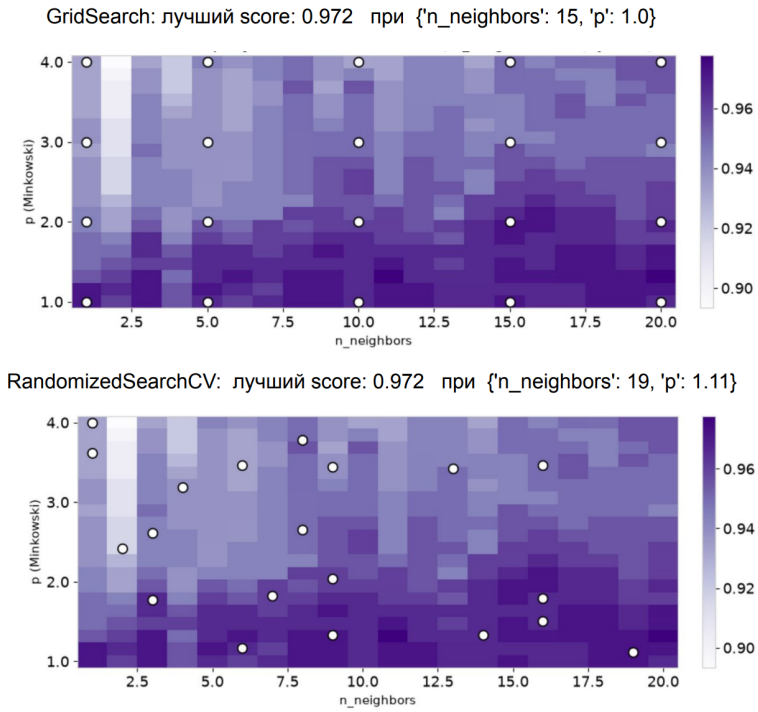


Рис. 21. Сравнение GridSearch и RandomizedSearch.

На фоне обеих картинок одна и та же тепловая карта качества:

- тёмный цвет — соответствует высокому значению метрики;
- светлый цвет — низкому.
- кружки поверх — это точки, которые алгоритм реально проверил, то есть обучил модель и замерил качество на кросс-валидации.

В этом конкретном примере оба метода нашли близкий по качеству результат (score $\approx 0{,}972$), но с разными комбинациями параметров. Это хорошо демонстрирует, что при большом пространстве поиска случайный перебор часто находит область не хуже сеточного, но заметно быстрее.

4.4. OPTUNA

Современные автоматические методы подбора гиперпараметров сфокусированы не только на достижении лучшего значения метрики. Сейчас их можно рассматривать как полноценную часть оценки и контроля качества модели. Такой грамотный подбор влияет:

- на обобщающую способность модели;
- на надёжность результатов (через воспроизводимость);
- на интерпретируемость.

В современных задачах разработчики чаще всего не полагаются на ручной подбор или простые методы вроде grid search, а используют более продвинутые и гибкие библиотеки, например optuna.

ОПРЕДЕЛЕНИЕ

Optuna — это современный фреймворк для автоматического подбора гиперпараметров, построенный на принципах:

- байесовской оптимизации;
- дерева Парзена (TPE);
- early stopping (pruning) для ускорения;
- поддержки мультицелевой оптимизации.

Optuna умеет:

- оптимизировать любые модели (не только из sklearn);
- работать с распределёнными экспериментами — одновременно запускать нескольких проб (trials) на разных машинах, ядрах или даже в разных docker-контейнерах;
- визуализировать кривые обучения;
- встраиваться в любые пайплайны — например, PyTorch, LightGBM, CatBoost или XGBoost.

Примечание. Вместо простого перебора optuna использует вероятностную модель Tree-Structured Parzen Estimator (TPE). Её особенности:

- строит аппроксимации успешных и неудачных конфигураций параметров.
- максимизирует ожидаемое улучшение (Expected Improvement).
- позволяет обоснованно выбирать следующие параметры без временных трат на нерелевантные участки пространства.

В результате даже 10–30 итераций могут дать более быстрый и чёткий результат по сравнению с перебором сотен комбинаций в grid search.

В примере ниже показана ранняя остановка обучения XGBoost. Аналогичный механизм есть в Optuna и называется **pruning**: он позволяет досрочно прекращать неудачные попытки подбора гиперпараметров

```python
# Импорт библиотеки
import optuna

def objective(trial):
    # Инициализируем сетку параметров
    n_estimators = trial.suggest_int("n_estimators", 50, 300) # trial.suggest_int указывает, что необходимо подобрать целое число в заданном диапазоне
    max_depth = trial.suggest_int("max_depth", 2, 10)

    model = XGBClassifier( # Создаём экземпляр модели XGBoost, передавая в неё текущие значения гиперпараметров
        n_estimators=n_estimators,
        max_depth=max_depth,
        eval_metric="logloss" # При обучении модель будет отслеживать логарифмическую функцию потерь
    )
    model.fit(
        X_train, y_train,
        eval_set=[(X_valid, y_valid)],
        early_stopping_rounds=20, # Если за 20 итераций log loss не улучшится, обучение остановится раньше
        verbose=False, # Глушим логи XGBoost
    )

    preds = model.predict_proba(X_valid)[:, 1]
    return roc_auc_score(y_valid, preds) # Возвращаем ROC-AUC — метрику качества, которую Optuna будет максимизировать
```

Иногда оптимизируется не только точность, но и, например, время инференса. Optuna поддерживает **Pareto-фронты** — точки, которые не хуже одновременно по всем метрикам.

```python
def objective(trial):
    max_depth = trial.suggest_int("max_depth", 2, 10)
    gamma = trial.suggest_float("gamma", 0.0, 1.0)

    model = XGBClassifier(max_depth=max_depth, gamma=gamma)
    start = time.time()
    model.fit(X_train, y_train)
    end = time.time()

    pred = model.predict(X_valid)
```

```
    acc = accuracy_score(y_valid, pred)
    latency = end - start
    return acc, latency  # Оптимизируем и точность, и задержку
```

Optuna умеет работать с вложенными параметрами и даёт возможность использовать условные ветвления. Например, если мы используем алгоритм SVM, `C` примет значение от 0,01 до 100.

```
def objective(trial):
    classifier_name = trial.suggest_categorical("classifier", ["SVM",
     ↪  "RandomForest"])
    if classifier_name == "SVM":
        c = trial.suggest_float("svm_c", 1e-5, 1e2, log=True)
        model = SVC(C=c)
    else:
        n_estimators = trial.suggest_int("rf_n_estimators", 10, 100)
        model = RandomForestClassifier(n_estimators=n_estimators)
    model.fit(X_train, y_train)
    return accuracy_score(y_valid, model.predict(X_valid))
```

А ещё Optuna поддерживает встроенные дашборды визуализации, которые помогают сделать процесс подбора более наглядным:

- optuna.visualization — importance, contour, optimization history;
- optuna-dashboard — web-интерфейс;
- возможна интеграция с TensorBoard и weights & biases.

> **Примечание.** Optuna — комплексная библиотека со множеством возможностей. Чтобы раскрыть её потенциал, нужно изучить документацию и адаптировать инструмент под конкретный случай.

4.4.1. Запуск Optuna

```
study = optuna.create_study(direction="maximize")
```

Создаём объект study, в котором Optuna будет сохранять результаты всех попыток. Параметр `direction = "maximize"` указывает на то, что необходимо максимизировать возвращаемую метрику: в данном случае ROC-AUC. Можно указать и `minimize`, но тогда алгоритм будет работать в другом направлении.

Если нужно, чтобы результат был воспроизводимым, можно зафиксировать seed.

```
study = optuna.create_study(direction="maximize",
 ↪  sampler=optuna.samplers.TPESampler(seed=42))
```

```
study.optimize(objective, n_trials=50)
```

Ставим задачу перед Optuna вызвать функцию `objective()` 50 раз, каждый раз подбирая новые гиперпараметры и сохраняя результат.

```
print("Лучшие параметры:", study.best_params)
print("Лучший ROC-AUC:", study.best_value)
```

После завершения работы Optuna:

- `study.best_params` — содержит словарь с наилучшими найденными гиперпараметрами;
- `study.best_value` — соответствующее значение метрики (например, ROC-AUC).

```
optuna.visualization.plot_optimization_history(study).show()
optuna.visualization.plot_param_importances(study).show()
```

Эти графики покажут:

- историю оптимизации, поясняя, как менялся результат после нескольких итераций;
- важность гиперпараметров, уточняя, как сильно каждый параметр повлиял на итоговую метрику.

Глава 5. Работа с моделями в sklearn

Пора вернуться на шаг назад — к самому процессу разработки решения. Вспомним, о чём мы говорили в рамках курса «Введение в искусственный интеллект». Точность модели зависит не только от выбранного алгоритма, но и от того, как ты работаешь с данными, разбиваешь их на выборки, масштабируешь и кодируешь признаки. Неправильная реализация этих шагов как со стороны логики происходящего, так и с точки зрения кода может привести к переобучению, утечке данных и нерелевантно высоким метрикам на валидации.

Чтобы избежать таких проблем и выстроить надёжный и воспроизводимый процесс, стоит сфокусироваться:

- на ключевых элементах scikit-learn (`fit`, `predict`, `predict_proba`);
- стратегии подготовке данных;
- построении pipeline, который станет скелетом экспериментов (это самый важный шаг).

Этот раздел послужит «мостиком» от идеи решения к её реализации.

5.1. API SKLEARN-МОДЕЛЕЙ: FIT, PREDICT, PREDICT_PROBA

При работе с классическими ML-функциями мы чаще всего используем библиотеку scikit-learn (sklearn). Sklearn реализует множество алгоритмов и функций, придерживаясь строгого интерфейсного единства. Это делает работу с библиотекой удобной, интуитивной и предсказуемой.

Вспомним ключевые методы:

- $fit(X, y)$ — обучает модель на тренировочных данных;
- $predict(X)$ — получает предсказания (индексы классов для классификации, числовые значения для регрессии);
- $predict_proba(X)$ — получает вероятностные предсказания для задач классификации, то есть модель отдаёт не предсказанные индексы, а вектор вероятностей. Эта функция может быть ключевым элементом для построения ROC-кривых, подсчёта AUC и работы с порогами.

ВАЖНО

Не все модели реализуют `predict_proba`. Например, в SVC (SVM) она доступна только при `probability=True`, что добавляет вычислительные издержки.

5.2. TRAIN/TEST SPLIT

Любое разбиение, которое мы описывали выше, можно получить комбинацией train_test_split из sklearn.model_selection. Здесь важно помнить

- что стратификация (`stratify = y`) обязательна в задачах классификации для сохранения распределения классов;
- фиксированный `random_state` обеспечивает воспроизводимость результатов.

5.3. МАСШТАБИРОВАНИЕ: STANDARDSCALER VS MINMAXSCALER

Многие модели машинного обучения чувствительны к масштабу входных признаков. Контролировать и модифицировать масштаб можно через специальные `scaler`-функции из `sklearn.preprocessing`.

StandardScaler

- Центрирует данные (среднее = 0) и масштабирует к единичной дисперсии (std = 1).
- Подходит, если признаки имеют близкое к нормальному распределение.
- Сохраняет выбросы, не ограничивая значения в каком-либо диапазоне.
- Часто используется по умолчанию для линейных моделей — например, PCA или SVM.

MinMaxScaler

- Масштабирует значения в диапазоне $[0, 1]$ или другом, заданном параметром `feature_range`.
- Подходит, если признаки равномерно распределены или когда важен абсолютный диапазон значений.
- Чувствителен к выбросам — экстремальные значения могут подавлять остальные.
- Уместен, если требуется интерпретируемый масштаб. Например, для изображений: пиксели $\rightarrow [0, 1]$.

RobustScaler

- Использует медиану и интерквартильный размах (IQR) и менее чувствителен к выбросам.
- Корректно работает при наличии экстремальных значений, не искажая остальные.
- Масштабирует данные так, чтобы медиана стала равна 0, а IQR — примерно 1.

ВАЖНО

При масштабировании необходимо обучать `scaler` только на тренировочной выборке. Затем полученные параметры (например, среднее и стандартное отклонение) применяются для трансформации как тренировочных, так и тестовых данных. Нарушение этого правила — частая причина утечки данных, когда информация из теста случайно «подсказывает» модели, как улучшить прогноз.

5.4. КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

Многие признаки категориальны, поэтому перед передачей в модель в большинстве случаев их нужно преобразовать в числовой вид (за исключением моделей, которые сами умеют обрабатывать подобные признаки). Однако способ кодирования зависит от контекста задачи и структуры признака.

- OneHotEncoder
  - Преобразует каждое уникальное значение категориального признака в отдельную бинарную колонку (0 или 1).
  - Применяется, если порядок между категориями не имеет смысла (например, «красный», «синий» или «зелёный»).
  - Может сильно увеличивать размерность набора данных при большом числе уникальных значений.

- OrdinalEncoder
  - Присваивает каждой категории целое число $(0, 1, 2, ...)$.
  - Используется, только если между категориями есть логический порядок (например, размер S < M < L).
  - Не подходит, если порядок не значим — это может ввести модель в заблуждение.
- LabelEncoder
  - Преобразует категории в целые числа как OrdinalEncoder, но не поддерживает вход из нескольких столбцов.
  - Часто используется для кодирования целевой переменной в задачах классификации.
  - Не рекомендуется для признаков, поскольку модель может ошибочно интерпретировать числа как порядковые значения.

5.5. ПОСТРОЕНИЕ PIPELINE

ОПРЕДЕЛЕНИЕ

Pipeline — это инструмент scikit-learn, объединяющий все шаги предобработки и обучения модели в единую структуру:

- предотвращает data leakage из теста в обучение;
- повышает воспроизводимость и делает код читаемым и масштабируемым;
- упрощает интеграцию с GridSearchCV, cross_val_score и другими инструментами автоматической оценки и подбора гиперпараметров.

Рассмотрим несколько pipeline-примеров.

```
# Импорт необходимых библиотек
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

# Названия признаков по типу данных
numeric_features = ['age', 'income']
categorical_features = ['gender', 'city']

# Предобработка: масштабирование числовых признаков + OneHot-кодирование категориальных
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),              # StandardScaler для числовых
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)  # OHE для категориальных
])

# Итоговый pipeline: предобработка   классификатор
clf_pipeline = Pipeline([
```

```
    ('preprocessing', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42)) # Не забыли про
     ↪  random_state для воспроизводимости
])

# Обучение пайплайна на тренировочных данных
clf_pipeline.fit(X_train, y_train)
```

Все шаги обёрнуты в `pipeline`, что предотвращает утечку данных: статистики `StandardScaler` и `OneHotEncoder` обучаются только на тренировочных данных во время `.fit()`.

```
from sklearn.linear_model import LogisticRegression

numeric_features = ['height', 'weight', 'age']

pipeline = Pipeline([
    ('scaler', StandardScaler()),  # Масштабируем все числовые признаки
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train[numeric_features], y_train)
```

Подходит для задач, в которых нет категориальных признаков. Масштабирование через стандартизацию часто помогает сходимости и улучшению качества логистической регрессии.

```
from sklearn.ensemble import HistGradientBoostingClassifier

categorical_features = ['product_type', 'region']

pipeline = Pipeline([
    ('encoder', OneHotEncoder(handle_unknown='ignore')),  # Кодируем категориальные
     ↪  признаки
    ('classifier', HistGradientBoostingClassifier())
])

pipeline.fit(X_train[categorical_features], y_train)
```

Хотя бустинг часто может сам работать с категориальными признаками, HistGradientBoostingClassifier в scikit-learn требует предварительного преобразования, только если не используется параметр `categorical_features`.

Одна из сильных сторон `pipeline` — его можно напрямую передавать:

- в `cross_val_score(pipeline, X, y, cv=5)` без ручного разделения данных;
- в `GridSearchCV(pipeline, param_grid, cv=5)`, в котором параметры указываются через двойное подчёркивание (например, `param_grid = {'classifier__n_estimators': [100, 200]}`).

Тот же подход, что и для `GridSearchCV`, можно реализовать с `pipeline` и для Optuna, Hyperopt и других библиотек для автоматизированного подбора гиперпараметров.

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(random_state=42))
])

def objective(trial):
    n_estimators = trial.suggest_int('classifier__n_estimators', 50, 300)
    max_depth = trial.suggest_int('classifier__max_depth', 2, 20)

    pipeline.set_params(
        classifier__n_estimators=n_estimators,
        classifier__max_depth=max_depth
    )

    return cross_val_score(pipeline, X, y, cv=3, scoring='accuracy').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
```

Такая гибкость `pipeline` даёт:

- чистый и модульный код,
- минимизацию ошибок валидации,
- автоматизацию и масштабируемость ML-процессов.

Материалы для более углублённого погружения в тему

- TimeSeriesSplit
- Байесовская оптимизация
- Дерево Парзена (TPE)
- Мультицелевая оптимизация
- Ожидаемое улучшение (Expected Improvement)
- Pipeline

Если кратко

- → Существует множество типов контроля качества ML-моделей. Методы оценки чаще объединяют одним общим названием — кросс-валидация.
- → Основные виды кросс-валидации: hold-out, k-Fold, Leave-One-Out, ShuffleSplit, Stratified/group CV, nested k-Fold.
- → Разбиением можно управлять независимо по трём осям: перемешиванием (Shuffle), группировкой (Group) и стратификацией (Stratify) — они комбинируются друг с другом (например, с помощью StratifiedGroupKFold).
- → Гиперпараметры — параметры модели, которые задаются экспертами до обучения.
- → Основа контроля качества — разбиение данных на несколько частей. Для подбора гиперпараметров — минимум на 3 части.
- → Бутстрэп — способ формирования новых выборок из существующих данных.
- → Кросс-валидация на временных рядах должна быть реализована так, чтобы не допустить утечек из будущего в прошлое.
- → Для подбора гиперпараметров существуют автоматизированные системы. На практике рекомендуется использовать современные методы, например Optuna.